
Refine — User Guide

A walkthrough of every page in the Refine application. The inventory and account pages cover all three clouds — AWS accounts, Azure subscriptions, and GCP projects — not just AWS.

Table of Contents

1. [Dashboard](#)
 2. [Cost Overview](#)
 3. [Commitments \(Savings Plans\)](#)
 4. [Savings Tracking \(Savings Report\)](#)
 5. [Savings Configuration](#)
 6. [Recommendations](#)
 7. [Exemptions & Resource Policies](#)
 8. [Cost Cleanup](#)
 9. [Cloud Accounts](#)
 10. [Inventory Pages](#)
 - [EC2 Instances](#)
 - [RDS Databases](#)
 - [S3 Buckets](#)
 - [EBS Volumes](#)
 - [EBS Snapshots](#)
 - [NAT Gateways](#)
 - [Load Balancers](#)
 - [Lambda Functions](#)
 - [CloudWatch Logs](#)
 - [Fargate Tasks](#)
 - [ElastiCache](#)
 - [Data Transfer](#)
 - [Batch Jobs](#)
 - [Lightsail](#)
 11. [User Management](#)
 12. [System Events](#)
 13. [Common Features](#)
 14. [Cost Allocation](#)
 15. [Usage Trends](#)
 16. [Directory Settings](#)
-

Where to Run Refine

Refine runs anywhere Docker is available. Choose based on your needs:

Option	Best For	Setup
Your laptop/desktop	Evaluation, demos, 1-5 AWS accounts	Extract ZIP, run <code>setup.bat</code> (Windows) or <code>./setup.sh</code> (Mac/Linux)
AWS EC2 instance	Production use, 5+ accounts, team access	Launch EC2, upload ZIP, run <code>./setup.sh</code>

Running locally is the fastest way to get started — under 5 minutes. For always-on monitoring or 5+ accounts, move to an EC2 instance. See [DEPLOYMENT GUIDE.md](#) for full instructions and [SYSTEM REQUIREMENTS.md](#) for sizing.

Local limitation: The nightly data sync only runs when your machine is on. If your laptop sleeps overnight, data won't refresh until you restart Refine. For always-on monitoring, use an EC2 instance.

Your First Week with Refine

Day 1 — Setup

- ☐ Run the setup wizard (`setup.bat` on Windows, `./setup.sh` on Mac/Linux)
- ☐ Connect your first AWS account (see [AWS MARKETPLACE SETUP GUIDE.md](#))
- ☐ Verify the Dashboard loads and shows "Sync scheduled"
- ☐ Check the **AWS Accounts** page — your account should show **Validated**

Days 2–7 — Data Collection

- ☐ The nightly sync runs automatically at 2 AM UTC. Your first full dataset will appear after the first sync completes
- ☐ EC2, RDS, and S3 data should appear within 24 hours
- ☐ Cost data may take 24–48 hours (depends on your S3 bucket population)
- ☐ **Don't worry about empty recommendation pages yet** — recommendations need 14+ days of CloudWatch metrics

Week 2 — Recommendations Start

- ☐ Rightsizing recommendations will begin appearing for EC2 and RDS instances with 14+ days of data
- ☐ Review the **Recommendations** page — each recommendation shows CPU/memory utilization and a confidence score
- ☐ Go to **Savings Config** to set your preferences (commitment term, payment options)
- ☐ Optionally configure the monthly savings email report

Week 4+ — Baselines Lock

- ☐ Resources tracked for 30+ days become "baselined" — savings calculations begin

- ☐ New cost optimization tools are available: Orphaned Resource Scanner, RI Analyzer, S3 Lifecycle Generator, Spot Scorer
- ☐ Review the **Savings Report** for your first monthly summary

Common Questions

Question	Answer
Why are some recommendations grayed out?	Resources under 30 days old are excluded from savings calculations
Why do I see \$0 in Cost Overview?	Cost data takes 24–48 hours after first sync. Check that your S3 bucket has data
Why zero RDS recommendations?	RDS needs 14+ days of CloudWatch metrics. Check back after that period
What does "Partial sync" mean?	Some accounts synced but others failed. Check System Events for details
How do I check if sync is working?	Go to AWS Accounts — the "Last Synced" column shows the timestamp

1. Dashboard

URL: `http://localhost:8000/dashboard`

The dashboard is your landing page after login. It gives a high-level overview of your entire AWS environment in one screen.

Sync Status Banner

If data collection has failed or data is stale, a warning banner appears at the top of the dashboard showing the sync status and last successful sync time. Click the banner for details or to trigger a manual sync.

Hero Metrics

Four cards across the top:

Metric	What It Shows
Total Monthly Cost	Estimated monthly spend across all connected AWS accounts
Total Resources	Count of all discovered resources across 14 AWS services
Potential Savings	Total monthly savings if all recommendations are applied. Once a savings baseline exists, this card shows Actual Savings — the realized cost reduction compared to your baseline month
Optimization Score	0–100 score based on how much waste exists (100 = fully optimized)

Cost Trend

A 12-month bar chart showing your monthly AWS spend over time. Hover over any bar to see the exact dollar amount. The current month is highlighted.

Cost by Service

A horizontal bar chart of your top services ranked by cost. Shows the service name and dollar amount for each.

Resource Inventory

A grid of 14 service tiles. Each tile shows:

- Service icon and name
- Resource count
- Monthly cost (if non-zero)

Click any tile to navigate to that service's detail page.

Top Recommendations

The highest-savings optimization opportunities across all services. Each entry shows the resource name, service type, recommendation action, and estimated monthly savings. Click any recommendation to go to the relevant service page.

Active Savings Plans

Shows your current Savings Plan status. If no plans are active, a link to **Configure Savings** takes you to the Savings Configuration page.

Actions

- **Sync Now** — triggers an immediate data refresh from your AWS accounts
- **Add Account** — opens the AWS account onboarding flow

2. Cost Overview

URL: `http://localhost:8000/costs`

A detailed breakdown of your AWS costs by service over the last 7 days.

Summary Cards

Card	Description
Total Cost	Sum of all service costs (updates when you search)
Services Tracked	Number of distinct AWS services with costs
Top Service	The service with the highest cost

Cost Chart

A horizontal bar chart of the top 10 services by cost. Each bar shows the service name and total cost.

Service Cost Table

A searchable, sortable table with columns:

Column	Description
Service	AWS service name
Cost	Total cost in USD
% of Total	Percentage of total spend

Features:

- **Search** — filter by service name (summary cards update live)
- **Sort** — click any column header to sort ascending/descending
- **Refresh Costs** — triggers a fresh data pull from AWS

How Refine Prices Compute

Every EC2 dollar figure — spend, eligible spend, coverage, Savings Plan sizing, rightsizing and commitment savings — is priced at AWS's on-demand list rate for the instance's **region and platform**. Windows and SQL Server AMIs are billed the Linux rate plus AWS's licence fee per vCPU-hour (Windows Server \$0.046/vCPU-hour, prorated to the baseline CPU share for burstable T sizes; SQL Server Web, Standard and Enterprise on top, with a four-vCPU minimum), read from the platform AWS reports on the instance. Instances collected by a stack older than template **v2.34.0** are re-priced on read; their costs rise to the correct figure after updating Refine. Bring-your-own-licence AMIs report a plain Windows or Linux platform and are priced as such; RHEL and SUSE subscriptions are not modelled and read as Linux.

3. Commitments (Savings Plans)

URL: `http://localhost:8000/savings-plans`

Analyzes your current EC2, RDS, Lambda, and Fargate spend and recommends optimal Savings Plan purchases.

Active Savings Plans

The Active Savings Plans table displays all Savings Plans and Reserved Instances you have purchased. Each plan shows:

Column	Description
Type	Compute SP, EC2 Instance SP, Database SP, or RDS RI
Family / Class	Instance family and region (for EC2 Instance SPs) or RDS class (for RDS RIs). Blank for Compute/Database SPs
Hourly	Hourly commitment rate (\$/hr), to the tenth of a cent AWS uses (e.g. \$0.116/hr)
Monthly	Monthly commitment (hourly x 730 hours)
Payment	No Upfront, Partial Upfront, or All Upfront
Term	1 year or 3 years
Start / End	Commitment period dates
Status	Active (in effect), Queued (future start date), or Expired
Utilization	Share of the commitment actually used, as reported by AWS. Green (95% or more), yellow (80–94%), red (below 80%). AWS reports utilization for all your Savings Plans together, so a figure appears here only when you have a single active Savings Plan (or a single RDS Reserved Instance); otherwise see Overall Utilization above

***Tip:** Add your active plans after purchasing them so Refine can monitor utilization and warn you before recommendations would cause underutilization.*

Current State

Four metric cards. When your account's CloudFormation stack collects commitment data (template **v2.33.0 or later**), every figure comes from **AWS Cost Explorer** and is labelled with the period it covers. On template **v2.34.0 or later** that is the **last 30 days** (a sliding window ending yesterday; a plan bought more recently shows the days it has existed); on v2.33.0 it is the current month to date once it spans a week, otherwise the last complete month:

Card	What it shows
Overall Utilization	Share of your Savings Plan commitment that was used during the period
Coverage Rate	Share of <i>all</i> Savings Plan–eligible spend (every EC2 family and region, Fargate, Lambda) that ran through the plans, shown in dollars, plus the most the plan could cover at 100% utilization — a small plan tops out well below 100% however well it is used. It also shows the coverable share: the part of your spend that runs every hour, which is the most any plan can cover without losing money, and how much of <i>that</i> your plans cover. For a fleet switched off part of the time the coverable share is small or zero, and a low coverage figure is then the correct outcome, not a shortfall
Monthly Commitment	Your active plans' hourly commitment x 730 hours, plus the commitment actually billed in the period
Net Savings	What the plans saved versus on-demand pricing, after paying for the commitment. Can be negative

A Savings Plan is billed every hour of its term, whether or not eligible usage runs. If covered instances are stopped overnight or at weekends, utilization drops (to 0% in those hours) and the plan can cost more than on-demand would

have. When utilization is below 95% or net savings are negative, a banner above the table shows the unused commitment and the loss; with daily data it also shows **weekday vs weekend utilization** and how many days the plan sat unused, and a bar per day appears above the Utilization by Month table (weekends shaded).

Why utilization and coverage can both be low. Utilization is about *hours*: the share of the commitment's hours that had usage. Coverage is about *spend*: the share of all Savings-Plans-eligible spend (every EC2 family and region, Fargate, Lambda) that ran through a plan. A fleet that is switched off at night and at weekends but busy in working hours produces both at once: the plan sits idle most hours (low utilization), and in the busy hours usage is far above what the plan covers, so most spend is billed on-demand (low coverage). The fix is usage that runs *every* hour in the plan's family and region, not a bigger plan. **Judge coverage against the coverable share, not against 100%:** only spend that runs every hour can be covered without losing money, so a fleet that is off at night can never show high coverage — and should not.

If your stack doesn't collect commitment data yet, the cards show **estimates** instead and say so: utilization assumes covered resources run 24/7, and **Est. Monthly Savings** assumes the full commitment is used. Estimates can read 100% utilization when AWS reports far less, so update the stack (Accounts page) and click **Sync Now** to see actual figures.

Utilization by Month

A month-by-month table of Savings Plan utilization, commitment billed, and net savings from AWS Cost Explorer, covering up to the last six months in which you had a commitment. The current month is marked (*to date*).

Spend Analysis

Breaks down your eligible monthly spend by service:

- EC2 Instances
- RDS Databases
- Fargate Tasks
- Lambda Functions
- Total Eligible Spend

Coverage Analysis

Per service (EC2, Fargate, Lambda): eligible monthly spend, how much of it the plans you **already own** cover today — estimated from their commitment, scaled by the utilization AWS reports — and how much more the recommended plan would add. The headline is AWS's own coverage figure for the account when it has been collected.

How Refine Sizes a Savings Plan

A Savings Plan commitment is billed **every hour** of its term, whether or not anything runs, and each hour is settled on its own: usage up to the commitment gets the discount, and commitment left over in that hour is lost — it never rolls into a busier hour. Refine therefore sizes every plan to be **100% utilized**, and lets you choose the **coverage**:

- **The commitment is the hourly floor.** Refine looks at every hour of the last 90 days and adds up the on-demand cost of everything running in the plan's scope (an instance family and region for an EC2 Instance Savings Plan; all EC2 for a Compute Savings Plan). The commitment is the level that was running in at least **99% of those hours** — whichever instances made it up. The 1% only absorbs a reboot or a late CloudWatch sample; it is not a part-time allowance.
- **It is a property of the fleet, not of any instance.** Ten part-time servers on the same 8-to-6 weekday schedule leave every night and weekend empty, so their floor is \$0. Ten part-time servers on staggered schedules that always keep something running have a real floor, and a plan sized to it is fully used even though not one of them is always on. The **hour-of-day chart** above the Commitment Breakdown shows the shape Refine read the floor from.

- **Coverage is your setting. Target Coverage** (Savings Configuration) is the share of eligible EC2 spend to cover. Refine commits the lower of that target and the floor, so a plan never covers more than what is always running. If your target is higher, the recommendation says it is capped and at what coverage. EC2 Instance Savings Plans are capped per instance family and region.
- **Savings Plans you already own are counted first.** Their coverage comes off the recommendation (an EC2 Instance Savings Plan within its own family and region, a Compute Savings Plan across everything), so Refine never recommends buying capacity you have already committed.
- **If AWS reports your current Savings Plans are less than fully used, Refine recommends no additional plan** and shows the utilization AWS reported. Adding commitment on top of commitment that is already going unused only increases waste.
- The commitment is rounded **down** to AWS's \$0.001/hour increment, so it never exceeds what is running.
- Instances with a rightsizing recommendation are committed at the smaller size, which is used in full whether or not you resize. An instance whose recommended size is in a **different family** is not committed to an EC2 Instance Savings Plan until it has moved. Idle instances are not committed.
- Fargate and Lambda are not committed, because Refine has no hourly usage data for them.
- **Stacks without hourly data** (an account-onboarding CloudFormation stack older than template **v2.34.0**) fall back to a per-instance rule: only instances that individually run at least 99% of hours over the last 30 and 90 days and every monthly snapshot are committed. It is deliberately conservative — it cannot see staggered schedules — and the page says which method was used. Update the stack for the full picture.

***Why "commit to 50% of an instance that runs 50% of the time" does not work.** The commitment is billed all 730 hours; the instance can only use it in the hours it runs. Halving the commitment halves what gets used too — utilization stays at 50% whatever the size, and below the plan's break-even (about 70% at a 30% discount) every size loses money. Only usage present in every hour can be committed to safely, which is why Refine sizes from the floor and not from averages.*

If nothing runs in every hour, Refine recommends **no purchase** and says why — including how many of the window's hours had nothing running — rather than committing to spend that won't be used.

Scenario Comparison

Side-by-side comparison of two EC2 savings strategies:

Scenario	Description
Compute Savings Plan	Covers EC2, Lambda, and Fargate with moderate discount. Most flexible — applies across instance families and regions
EC2 Instance Savings Plan	Covers EC2 only with higher discount. Committed to specific instance families and regions

Each scenario shows monthly savings, annual savings, hourly commitment, covered services, discount rate, and the coverage of EC2 spend it reaches at 100% utilization (noting when your target is capped). When no purchase is recommended — nothing runs in every hour, or AWS reports your existing plans under-used — the card says **None recommended** and why. The active scenario is highlighted. Click a scenario card to switch.

RDS Analysis

Depending on your configured RDS strategy:

- **Database Savings Plan** — shows hourly commitment, discount rate, and projected savings for generation 7+ RDS instances
- **Reserved Instances** — shows a comparison table of 1-year vs 3-year terms across No Upfront, Partial Upfront, and All Upfront payment options

Other Services

Shows Lambda and Fargate spend with a note that these are only covered by Compute Savings Plans.

Commitment Breakdown

A detailed table of spend and commitment, under the hour-of-day chart the floor was read from. The **EC2 Instance SP** tab lists each instance family **and region** (EC2 Instance Savings Plans are bought per both) with its uptime mix (always on, business hours, part-time, idle, unknown), how many instances are individually always on (plus "+ overlap" when part of the floor comes from part-time instances whose schedules overlap), the coverage reached (and the most that can be covered at 100% utilization), hourly commitment and monthly savings. The **Compute SP** tab shows the single cross-family plan. If EC2 rightsizing recommendations exist, checkboxes let you see how applying them would change your commitment; Refine re-sizes on the server for the selection and the table says "Recalculating..." until the figures arrive.

Alternative Plans

Plan cards comparing your current configuration against alternative options (different plan types, terms, and payment options). Click any card to apply that configuration.

Purchase Guide

Step-by-step instructions for purchasing. Includes a **Download Purchase Script** button that generates a `.sh` file with the exact AWS CLI commands for your recommended plan, reflecting the rightsizing checkboxes. When no EC2 purchase is recommended, the script and the guide say so and contain no Savings Plan purchase command. The script includes:

- Savings Plan offering lookup commands
- Purchase commands (commented out for safety — uncomment when ready)
- RDS Reserved Instance or Database Savings Plan commands (if applicable)

Important: Review the purchase script before running it. Purchases are billed immediately and cannot be reversed.

EC2 Rightsizing Table

Recommendations flagged by the commitment guard (they would leave an existing Savings Plan unused) start **unchecked** and carry a red note; check them only once your plans are fully used.

Shows EC2 instances that could be resized for cost savings. Columns include current type, recommended type, CPU utilization, current cost, and monthly savings. Checkboxes let you toggle recommendations on/off to see the impact on your Savings Plan commitment.

4. Savings Tracking (Savings Report)

URL: `http://localhost:8000/savings-report`

Tracks your actual realized savings over time by comparing your current AWS spend against a historical baseline. This page becomes available after Refine has collected at least two months of cost data.

Hero Cards

Card	Description
Total Saved	Monthly savings since the baseline date: resource optimization plus Savings Plan and Reserved Instance savings, when those can be attributed (see below). The line underneath splits it into its two parts.
Baseline Date	The month Refine began tracking savings against
Cost Reduction %	Percentage reduction in the estimated monthly cost of the resources tracked since baseline
Months Tracked	Number of months of savings data collected

Cost Trend Chart

A month-by-month line chart comparing your baseline monthly cost against actual spend. The shaded area between the two lines represents realized savings. Hover over any data point to see the exact baseline, actual, and saved amounts for that month.

Savings by Service

A table breaking down total savings by AWS service:

Column	Description
Service	AWS service name
Baseline Cost	Monthly cost at the baseline date
Current Cost	Most recent monthly cost
Monthly Savings	Difference between baseline and current
% Reduction	Percentage improvement

Savings Plans & Reserved Instances

Shows what AWS actually credited you for Savings Plans and Reserved Instances, taken from AWS Cost Explorer:

Figure	Description
Savings Plans	Net savings versus on-demand pricing. Can be negative if a plan is under-used.
Reserved Instances	Net savings versus on-demand pricing, across all RI types
Utilization	Share of your Savings Plan commitment that was actually used
Coverage	Share of eligible spend covered by Savings Plans, with the share that runs every hour (the most any plan can cover without loss) — the yardstick to judge it against

The headline uses the most recent **complete** month, so a month-to-date figure early in the month doesn't understate it. The card also lists your active Savings Plans.

What counts toward Total Saved. Only commitment savings that are new since your baseline. If you already had Savings Plans saving you money before Refine started tracking, that amount is subtracted, because those savings pre-date Refine. If Refine can't tell what was in place at your baseline, the figures are still shown but are **not** added to Total Saved.

***Not seeing this card's figures?** Commitment savings are collected by your account's onboarding CloudFormation stack, template **v2.33.0 or later**. Update the stack, then click **Sync Now** — up to 12 months of history is filled in on the first sync.*

How We Calculate This

Refine tracks two independent kinds of savings, and reports them separately:

- **Resource optimization** — Refine records each resource's estimated monthly cost when it is first tracked, then compares it with that resource's estimate today. These estimates use on-demand list prices, so they show the effect of rightsizing, deleting, or changing resources. Resources created after the baseline are excluded and shown separately as *New Resources Added*. If Refine's pricing of an unchanged resource changes after its baseline was recorded — the Windows and SQL Server licence portion added in v3.14.5, or an updated price table — the baseline is re-based to the new price basis (account-onboarding template **v2.35.0** or later), so a price change is never reported as growth or as a saving.
- **Savings Plans and Reserved Instances** — a commitment doesn't change a resource's list price, so it can't show up in the comparison above. Refine takes these figures from the credits AWS actually applied to your bill.

Because the two are measured independently, adding them doesn't double-count. Neither figure is a projection from recommendations.

***Note:** Savings tracking requires the `SAVINGS_ENABLED` configuration and an S3 bucket in your own AWS account for storing cost snapshots. No data is sent to Blacktip. See the setup guide for details.*

Export

- **Download Report (PDF)** — generates a formatted PDF summary of your savings to date, suitable for sharing with stakeholders
- **Export CSV** — downloads the summary, per-service breakdown, top savings resources, your Savings Plans, and month-by-month history. The summary lists resource optimization savings, Savings Plan and Reserved Instance savings, and the combined total on separate rows, and states whether commitment savings are included in the total.

5. Savings Configuration

URL: `http://localhost:8000/savings-config`

Configure your savings optimization preferences. These settings control how Refine calculates recommendations on the Savings Plans page.

Plan Type

Choose between:

- **Compute Savings Plan** — covers EC2, Lambda, and Fargate (most flexible)
- **EC2 Instance Savings Plan** — covers EC2 only (higher discount)

Commitment Term

- **1 Year** — lower commitment, smaller discount
- **3 Years** — higher commitment, larger discount

Payment Option

- **No Upfront** — pay monthly, smallest discount
- **Partial Upfront** — pay some upfront, medium discount
- **All Upfront** — pay everything upfront, largest discount

Savings Plan Target Coverage

A slider (10–100%, default 80%) that sets the share of your eligible EC2 spend to cover with a Savings Plan. Plans are always sized to be 100% utilized, so the recommendation is capped at the hourly floor — what runs in every hour; if that is less than your target, the Savings Plans page shows the coverage actually reached. See [How Refine Sizes a Savings Plan](#).

Rightsizing Target Utilization

A slider (50–100%, default 80%) for the CPU and memory utilization that rightsizing recommendations aim for. Lower leaves more headroom; higher packs workloads tighter. It does not affect Savings Plan sizing.

Instance Family Strategy

Three options for EC2 Instance Savings Plans:

Strategy	Description
Single Family	Commit to one instance family (e.g., m6i) — simplest approach
Multi-Family	Select multiple preferred families — Refine optimizes across them
Auto-Optimize	Refine automatically selects the best families based on your workload

When Single or Multi-Family is selected, a categorized family picker appears with tabs: All, General Purpose, Compute Optimized, Memory Optimized, Storage Optimized, Accelerated Computing, High Memory. Each family shows a description and use case.

Force Family Migration

A toggle that tells the optimizer to migrate instances from non-preferred families to your preferred families, even if there is no immediate cost savings. Useful for standardizing your fleet.

RDS Savings Strategy

- **Reserved Instances** — commit to specific instance classes for maximum discount (up to 63% with 3-year all-upfront)

- **Database Savings Plan** — flexible commitment that applies to generation 7+ instances across engines and regions (~20% off)

RDS Instance Classes

Select your preferred RDS instance classes (e.g., db.m6i, db.r6i, db.t3) organized by category.

S3 Data Retention

Number of days to retain S3 data before recommending lifecycle policies. Default: 365 days.

EBS Snapshot Retention

Number of days to retain EBS snapshots before recommending deletion. Default: 90 days.

Savings Breakdown

A real-time summary panel showing projected savings based on your current configuration:

- EC2 Savings
- RDS Savings
- Storage Savings
- Total Monthly Savings

Budget Template

Download a CloudFormation template that creates AWS Budget alerts based on configurable thresholds.

Optimizer Thresholds

The **Optimizer Thresholds** tab lets you tune *how aggressively* Refine flags resources for idle/rightsizing recommendations — without touching code. Each resource class (Compute, Databases, Serverless, Block Storage, Object Storage, Caching, Containers) has its own card with:

- **Preset selector** — *Conservative, Default, or Aggressive*. Default matches Refine's built-in cutoffs, so changing nothing keeps today's behavior.
- **Advanced overrides** — expand to edit the raw knobs (e.g. Compute's *CPU avg %*, *CPU peak %*, *Memory avg %*, *Network floor*). Editing any knob flips the card to *Custom*.
- **Reset to preset** — discards your overrides and falls back to the canned default.

Thresholds are **shared across clouds per resource class** — the Compute card governs EC2, Azure VMs, and GCE alike. Lower idle cutoffs flag *fewer* resources; raising them (Aggressive) surfaces *more* rightsizing/idle wins. Changes take effect on the next sync.

*Example: sliding **Compute** to Aggressive raises the idle CPU cutoff, so a VM averaging 8% CPU that was previously "oversized" is now flagged "idle — stop or delete."*

6. Recommendations

Move into your Savings Plan (`migrate_into_commitment`): when an EC2 Instance Savings Plan you own sits idle in some hours, Refine looks for instances in *other* families that run in exactly those hours — a nightly batch box, say — and

proposes the smallest size in the plan's family that fits its vCPUs and memory. The saving is priced hour by hour: in each hour the instance runs, the plan absorbs what it can of the new size's cost on top of what the family already uses, and the rest stays on-demand. Instances whose hours coincide with the plan's busy hours are not proposed (the plan is already full then, and the new size may cost more). Windows workloads are never moved to Graviton, and moves to burstable sizes carry a note when the instance's average CPU is above the size's baseline. Needs hour-by-hour running data (template v2.34.0 or later).

URL: `http://localhost:8000/recommendations`

A consolidated view of all optimization recommendations across every AWS service.

Summary Cards

Card	Description
Total Monthly Savings	Sum of all recommendation savings (filtered view). Recommendations that would leave an existing Savings Plan or Reserved Instance partly unused are shown but not counted — see Existing Commitments Come First
Total Annual Savings	Monthly savings x 12
Total Recommendations	Count of recommendations (filtered view)
High Priority	Count of high-priority recommendations

Recommendations Table

Column	Description
Priority	High (red), Medium (amber), or Low (green) badge
Confidence	High, Medium, or Low badge — based on CPU/memory utilization data availability and consistency
Service	AWS service type (EC2, RDS, S3, etc.)
AWS Account	Account ID where the resource lives
Resource	Resource name and ID
Recommendation	What to do and why
Change	Current → Recommended (e.g., <code>t2.large</code> → <code>t3.medium</code>)
Monthly Savings	Estimated dollar savings per month

Filters

- **Search** — filter by resource name, ID, description, or account ID
- **Priority** — filter by high, medium, or low
- **Service** — filter by AWS service type
- **Recommendation Type** — filter by action type (rightsized, delete, etc.)

All summary cards update live as you filter.

Actions

- **View CLI** — click on any recommendation row to see the AWS CLI command to implement it. A copy button lets you copy the command to your clipboard.
- **Export Script** — select recommendations with checkboxes and click **Export CLI** to download a `.sh` script with all selected CLI commands, grouped by service. The script covers all 14 supported service types (EC2, RDS, EBS Volumes, EBS Snapshots, S3, NAT Gateways, Load Balancers, Lambda, CloudWatch Logs, Fargate, ElastiCache, Data Transfer, Batch Jobs, and Lightsail). Files are named `refine-recommendations-{accountId}.sh` — one file per AWS account, each containing only that account's recommendations.
- **Exempt** — click the Exempt button on any recommendation to exclude a resource from optimization. You must select a reason (Contractual, Performance, Compliance, Temporary, Shared, Vendor, or Other) and can optionally add notes. Exempted resources move to the [Exemptions](#) page and their cost remains in Savings Plan commitment calculations, preserving SP utilization.

Note on idle recommendations: EC2 and RDS idle recommendations are generated even when the instance is already at the smallest available type (t3.micro / db.t4g.micro). In that case, the recommendation is to terminate the instance rather than downsize further.

Existing Commitments Come First

A Savings Plan, Reserved Instance, Azure reservation or GCP committed-use discount is billed for its whole term whether or not anything uses it. Stopping, downsizing or moving an instance that a commitment covers does not save that instance's cost — it leaves that much commitment unused. Refine prices every recommendation against your commitments:

- Each commitment covers a **scope**: an EC2 Instance Savings Plan covers one instance family in one region; a Compute Savings Plan covers all EC2, Fargate and Lambda; an RDS Reserved Instance covers one class in one region; a Database Savings Plan covers all RDS. Azure reservations cover a SKU in a region; GCP resource commitments a machine type in a region.
- Only usage **beyond** what the commitment covers saves money when removed. With hour-by-hour running data (template v2.34.0 or later) Refine works this out per instance, hour by hour: stopping a box that only runs while the plan is already full strands nothing, while stopping the one box that runs through the night strands exactly its night-time usage. Without hourly data, if AWS reports your Savings Plans less than fully used, nothing in scope is treated as removable; without AWS figures Refine estimates from inventory and says so.
- A recommendation that reduces covered usage — stop, terminate, downsize within the family — is **flagged** with a red note and shows its **net** figure (usually \$0), with the claimed figure struck through. One that moves the instance **out** of the commitment — another family, Graviton, Spot, another RDS class under an RI — also adds the new size at on-demand rates, so the net figure can be negative.
- Flagged recommendations are **excluded from every total** (Recommendations page, Dashboard, Savings Report pending savings, export script, per-service pages, the Savings Plans page's rightsizing table) and dropped to low priority. They stay visible so you know what not to do yet.
- The same rule applies to the Graviton, Spot and RI analysis tools: covered instances are not Spot candidates, Graviton moves off a covered family are blocked, and capacity an existing plan covers is not recommended for another commitment.
- Accepting or implementing a flagged recommendation shows the same warning.

The right order is: use the commitments you already pay for fully (the Savings Plans page shows utilization from AWS), then act on the recommendations that remain.

7. Exemptions & Resource Policies

URL: `http://localhost:8000/exemptions`

This page has two parts: **Exemptions** (recommendations you've dismissed) and **Resource Policies** (standing per-resource flags). Both keep resources you can't or won't change from cluttering your recommendations — and from skewing your savings totals.

Exemptions are resources you have intentionally excluded from optimization recommendations. Use exemptions when a resource cannot be changed due to contractual, performance, compliance, or other business reasons.

Summary Cards

Card	Description
Total Exemptions	Number of currently exempted resources
Monthly Savings Foregone	Total potential savings you are choosing not to pursue
Temporary	Count of exemptions that will auto-expire and resurface the recommendation

Exempted Resources Table

Column	Description
Resource	Resource name and ID
Service	AWS service type
Recommendation	The recommendation that was exempted
Reason	Why the resource was exempted
Notes	Additional context provided when exempting
Exempted	When the exemption was created
Expires	When the exemption auto-expires (temporary exemptions only), or "Never"
Foregone Savings	Monthly savings this exemption forgoes
Actions	Clear button to remove the exemption and resurface the recommendation

Filters

- **Reason** — filter by exemption reason (Contractual, Performance, Compliance, etc.)
- **Service** — filter by AWS service type

How Exemptions Work

1. **From Recommendations page:** Click the **Exempt** button in the Actions column, select a reason, and optionally add notes
2. **From any service page:** Click the **Exempt** button next to the savings amount for any resource with a recommendation

3. **Temporary exemptions:** Select "Temporary / Planned Change" as the reason to set an auto-expiry (default 90 days). The recommendation reappears after expiry.
4. **Clearing exemptions:** Click **Clear** on the Exemptions page to remove an exemption and resurface the recommendation
5. **Savings Plan impact:** Exempted resources keep their full cost in the SP commitment calculation, ensuring your Savings Plans stay fully utilized

Resource Policies

Below the exemptions table, the **Resource Policies** card lists every standing per-resource flag you've set across AWS, Azure, and GCP. Unlike an exemption (which dismisses one specific recommendation), a policy is a durable rule on the resource itself:

Flag	Effect
Permanent	Suppresses idle / stop / delete recommendations — the resource must stay up (e.g. contractual).
Size-locked	Suppresses downsize / rightsize / tier-change recommendations — the resource needs its exact size.
Exclude from commitments	Drops the resource from Savings Plan / Reserved Instance / CUD sizing, so a short-lived resource doesn't inflate a commitment.

Setting a policy: on any inventory page, use the ⚙️ **Assign** / ⚙️ **Edit** control in the *Policy* column. Flags are independent — set any combination, optionally with a note. Available on every inventory type (compute, databases, storage, serverless, containers, caching, network, logs) across all three clouds.

What you'll see when a policy blocks a recommendation: the inventory row still shows what the recommendation *would* have saved, but the savings figure is **struck through** with a 📌/🔒 "blocked" marker, and that amount is **excluded** from the page's Potential Savings total and Idle/Oversized count — so locking a resource never inflates your headline numbers.

Removing a policy: click **Remove** on this page, or open the ⚙️ control on the inventory row and clear the flags.

8. Cost Cleanup

URL: `http://localhost:8000/cost-cleanup`

Find resources by tag and generate CLI cleanup scripts.

How to Use

1. **Select a tag key** from the dropdown (e.g., `environment`, `project`, `team`)
2. **Select one or more tag values** from the checkboxes that appear (e.g., `dev`, `staging`)
3. Click **Find Resources**

Results

Refine queries all 10 supported resource types and shows:

Summary Cards:

- Resource Types found

- Total Resources matched
- Estimated Monthly Savings
- Annual Savings

Resource Groups — one section per resource type with matches:

Resource Type	Type-Specific Columns
EC2 Instances	Instance Type, State
RDS Databases	Engine, Instance Class
EBS Volumes	Volume Type, Size (GB)
EBS Snapshots	Volume Size, Age (days)
NAT Gateways	VPC ID, State
Load Balancers	Type, State
Lambda Functions	Runtime, Memory (MB)
Fargate Tasks	Cluster, vCPU
S3 Buckets	Size (TB), Object Count
CloudWatch Logs	Storage (GB), Retention (days)

Each group has a sortable table with checkboxes. Select the resources you want to clean up.

Download Cleanup Script

Click **Download Cleanup Script** to generate a `.sh` file with AWS CLI commands for all selected resources. The script includes:

- Commands grouped by resource type
- Safety warnings and notes (e.g., "delete database replicas first")
- Tag filter and timestamp in the header

Important: Always review the cleanup script and test in a non-production environment first.

9. Cloud Accounts

URL: `http://localhost:8000/accounts`

Manage the AWS accounts, Azure subscriptions and GCP projects connected to Refine. The sidebar calls this page **Cloud Accounts**; the AWS-specific columns and actions are described below, the Azure and GCP onboarding steps are in their own setup guides.

Accounts Table

Column	Description
Account Name	Friendly name for the account
Account ID	12-digit AWS Account ID
Region	AWS region where the CloudFormation stack is deployed
Status	Validated (green), Pending Setup (yellow), or Error (red)
Primary	Badge indicating the primary account
Last Synced	Timestamp of the last successful data sync

Adding an Account

1. Click **Add Account**
2. Enter your 12-digit AWS Account ID, an optional name, and select a region
3. Click **Add Account**
4. A setup panel appears with:
 - Your **External ID** (copy this for CloudFormation)
 - A **Download CloudFormation Template** button
5. Deploy the CloudFormation template in your AWS account (see the setup guide for details)
6. Choose an **Access Method**:
 - **Access key** (default) — paste the `RefineServiceAccessKeyId` and `RefineServiceSecretAccessKey` from the stack Outputs. Use this if Refine runs outside AWS (Azure, GCP, on-prem, a laptop), or if Refine and the monitored account are in different AWS partitions (commercial ↔ GovCloud).
 - **Keyless — this Refine server's IAM role** — no keys to paste. Requires Refine to run on EC2 in the **same AWS partition** as the monitored account, and the stack to have been deployed with the `RefineHostRoleArn` parameter set to that server's role ARN. In this mode the stack creates no IAM user, no access key, and no key outputs; its `CredentialMode` output confirms the mode.
7. Copy the **Role ARN** and **S3 Bucket Name** from the CloudFormation Outputs tab back into Refine
8. Click **Save** — Refine saves your configuration and automatically validates the connection (tests IAM role assumption and S3 bucket access). No separate "Validate" click is needed

Account Actions

- **Details** — view or edit account configuration
- **Set Primary** — mark an account as the primary account
- **Remove** — delete an account (requires confirmation)

Updating an Existing CloudFormation Stack

When Refine releases an update that includes Lambda changes, the AWS Accounts page will alert you automatically — no need to hunt through menus:

- A **yellow banner** appears at the top of the AWS Accounts page listing every account that needs updating

- Each row in the accounts table shows an **"Update CF ⚠"** button in the Actions column and a clickable amber badge in the CF Version column
- The **in-app changelog popup** (shown after a Refine upgrade) also includes a CF update callout with per-account status

To update a stack, click **Update CF ⚠** next to any account (or the CF version badge). This opens a focused update flow:

1. Click **Download Latest Template** to get the current `aws-setup.yaml`
2. In **AWS Console** → **CloudFormation** → select your Refine stack → **Update**
3. Choose **Replace current template** → upload the downloaded template
4. Click through all pages — existing parameters are preserved automatically
5. Acknowledge the IAM capabilities checkbox → **Update stack**
6. Wait for status **UPDATE COMPLETE**
7. Back in Refine, click **Validate Now** to confirm the update was applied

Updating the stack does **not** delete your S3 bucket or any collected data.

Multiple Accounts

You can connect multiple AWS accounts. Each gets its own CloudFormation stack. Use the AWS account selector in the sidebar to filter data by account across all pages.

10. Inventory Pages

The sidebar groups inventory into nine multi-cloud pages — **Compute, Databases, Storage, Block Storage, Caching, Serverless Functions, Container Workloads, Network** and **Logs** — each showing the AWS, Azure and GCP resources of that kind side by side, with cloud and account filters and the same summary cards, sortable table, exempt buttons and empty states as the AWS pages below. The per-service AWS pages documented in this section (EC2 at `/ec2`, RDS at `/rds`, and so on) hold the AWS-specific columns and recommendations and remain reachable by URL. All inventory pages share a common layout:

- **Summary cards** at the top with key metrics
- **Sortable, searchable data table** with service-specific columns
- **Filters** in the sidebar or table header
- **Refresh Inventory** button to trigger a fresh sync
- **Exempt button** on each resource with savings — click to exclude the resource from optimization recommendations (see [Exemptions](#))
- **Empty states** — when no data is available for a service, the page shows a helpful message explaining why (e.g., no resources of this type found, sync not yet completed) instead of a blank table

EC2 Instances

URL: `http://localhost:8000/ec2`

Summary Cards: Total Instances, Running Instances, Total Monthly Cost, Total Potential Savings

Summary Table — grouped by instance type showing count, running/stopped split, regions, and costs.

Details Table:

Column	Description
Name	Instance name tag
Account ID	AWS Account ID
Instance Type	e.g., t3.medium, m6i.large
State	Running, Stopped
CPU Avg / Peak	Average and peak CPU utilization across three periods (1-month, 3-month, 1-year) — each period has its own column group
Running Hours	Actual CloudWatch running hours for each period (1mo/3mo/1yr). Requires a CloudFormation stack update for existing accounts — see the AWS Accounts page for instructions
CPU Pattern	Stable, Variable, Spiky, or Idle (based on usage patterns)
Memory %	Memory utilization (if CloudWatch agent is installed)
Network	Network throughput
Monthly Cost	Estimated monthly cost
Rightsizing	Optimal, Oversized, Idle, or Terminate badge
Potential Savings	Estimated savings from applying the recommendation

Filters: Search (name/ID/type), Region, State, Instance Type, Rightsizing Status

Recommendations Panel: A summary of top rightsizing opportunities with estimated savings.

RDS Databases

URL: `http://localhost:8000/rds`

Summary Cards: Total Databases, Available Databases, Total Storage (TB), Total Monthly Cost, Total Potential Savings

Summary Table — grouped by engine (MySQL, PostgreSQL, Aurora, etc.) showing count, Multi-AZ instances, storage, and estimated Reserved Instance savings.

Details Table:

Column	Description
Name	DB instance identifier
Instance Class	e.g., db.m6i.large
Engine	MySQL, PostgreSQL, Aurora, etc.
Status	Available, Stopped
CPU Avg / Peak	CPU utilization (multiple periods)
CPU Pattern	Stable, Variable, Spiky, or Idle
Memory %	Memory utilization
Connections	Average connection count
Monthly Cost	Estimated monthly cost
Rightsizing	Optimal, Oversized, Idle badge
Potential Savings	Estimated savings

Filters: Search (name/ID), Engine, Status, Region, Instance Class

S3 Buckets

URL: `http://localhost:8000/s3`

Summary Cards: Total Buckets, Total Data (TB), Total Monthly Cost, Total Potential Savings

Summary Table — grouped by region showing bucket count, total size, object count, and counts of buckets with lifecycle policies and Intelligent-Tiering.

Details Table:

Column	Description
Bucket Name	Clickable link to bucket detail page
Region	AWS region
Size (GB)	Total storage size
Object Count	Number of objects in the bucket
Storage Classes	Badges showing which classes are in use (Standard, Glacier, IA, etc.)
Lifecycle Policy	Yes/No badge
Intelligent-Tiering	Yes/No badge
Monthly Cost	Estimated monthly cost
Recommendation	Add Lifecycle, Enable Intelligent-Tiering, Archive Old Data, or Optimal

Filters: Search (bucket name), Region, Lifecycle Policy, Intelligent-Tiering

Bucket Detail Page (/s3/{bucket_name}) — click any bucket name to see storage class breakdown, folder-level analysis, and bucket-specific recommendations.

Recommendations Panel: Highlights buckets without lifecycle policies, without Intelligent-Tiering, large buckets, and archival opportunities.

EBS Volumes

URL: http://localhost:8000/ebs-volumes

Summary Cards: Total Volumes, GP2 Eligible for Migration, Total Monthly Cost, Total Potential Savings

Details Table:

Column	Description
Volume ID	EBS Volume identifier
Volume Type	gp2, gp3, io1, etc. (gp2 highlighted as migration candidate)
Size (GB)	Volume size
IOPS	Provisioned IOPS
State	In-use, Available
Attached Instance	EC2 instance name if attached
Read/Write Ops	Average I/O operations (1-month)
Idle Time %	Percentage of time with no I/O
Monthly Cost	Estimated monthly cost
Recommendation	Keep, Change Type (gp2 → gp3), or Downsize
Potential Savings	Estimated savings

Filters: Search (volume ID/instance name), Region, Volume Type, Recommendation

***Tip:** gp3 volumes are up to 20% cheaper than gp2 with the same or better performance. Refine identifies all eligible gp2 volumes for migration.*

EBS Snapshots

URL: `http://localhost:8000/ebs-snapshots`

Summary Cards: Total Snapshots, Total Size (TB), Total Monthly Cost, Potential Savings

Details Table:

Column	Description
Snapshot ID	EBS Snapshot identifier
Description	Snapshot description
Volume ID	Source volume
Size (GB)	Snapshot size
Age (Days)	Days since creation (amber if >30d, red if >90d)
State	Completed, Pending
Encrypted	Yes/No badge
Monthly Cost	Estimated monthly cost
Recommendation	Keep, Delete, or Review
Reason	Why the recommendation was made

Filters: Search (snapshot ID/description/volume ID), Region, State, Encrypted, Recommendation

NAT Gateways

URL: `http://localhost:8000/nat-gateways`

Summary Cards: Total Gateways, Idle Gateways, Total Monthly Cost, Total Potential Savings

Details Table:

Column	Description
Gateway ID	NAT Gateway identifier
VPC ID	Associated VPC
Availability Zone	AZ where the gateway is deployed
Data Processed (GB)	Data transferred in the last month
Connection Count	Average connections (1-month)
Idle	Yes/No badge — idle if minimal traffic
VPC Endpoint Opportunities	AWS services that could use VPC endpoints instead of NAT
Orphaned EIP Cost	Cost of unused Elastic IPs associated with this gateway
Monthly Cost	Estimated monthly cost
Recommendation	Keep, Delete, or Add VPC Endpoints
Potential Savings	Estimated savings

Filters: Search (gateway ID/VPC ID), Region, Idle status

***Tip:** Each idle NAT Gateway costs ~\$32/month. VPC endpoints for S3 and DynamoDB are free and eliminate NAT Gateway data processing charges for those services.*

Load Balancers

URL: `http://localhost:8000/load-balancers`

Summary Cards: Total Load Balancers, Idle Load Balancers, Total Monthly Cost, Total Potential Savings

Details Table:

Column	Description
Name	Load balancer name
Type	ALB (Application) or NLB (Network) badge
Scheme	Internet-facing or Internal
State	Active, Provisioning
Request Count	Total requests in the last month
Healthy / Unhealthy Targets	Target group health status
Target Group Count	Number of associated target groups
Idle	Yes/No badge
Monthly Cost	Estimated monthly cost
Recommendation	Keep, Delete, or Consolidate
Potential Savings	Estimated savings

Filters: Search (LB name), Region, Type, Idle status

Lambda Functions

URL: `http://localhost:8000/lambda`

Summary Cards: Total Functions, Overprovisioned Count, Total Monthly Cost, Total Potential Savings

Details Table:

Column	Description
Function Name	Lambda function name
Runtime	Node.js, Python, Java, etc.
Memory (MB)	Allocated memory
Timeout (s)	Function timeout
Provisioned Concurrency	Count or "None"
Avg / Max Duration	Execution time in milliseconds
Max Memory Used	Peak memory consumption
Memory Utilization %	How much of allocated memory is used (color-coded)
Invocations	Total invocations (1-month)
Errors / Throttles	Error and throttle counts
Monthly Cost	Estimated monthly cost
Recommendation	Optimal, Downsize Memory, or Remove Provisioned Concurrency
Potential Savings	Estimated savings

Filters: Search (function name), Region, Runtime, Recommendation

CloudWatch Logs

URL: `http://localhost:8000/cloudwatch-logs`

Summary Cards: Total Log Groups, Total Storage (GB), Total Monthly Cost, Total Potential Savings

Details Table:

Column	Description
Log Group Name	CloudWatch Log Group name
Retention	Current retention period (color-coded: Never=red, ≤30d=green, 30-90d=amber, >90d=red)
Storage (GB)	Amount of log data stored
Write Frequency	Average writes per hour (1-month)
Last Event	Date of last log event
Unused	Yes/No badge — no recent events
Recommended Retention	Suggested retention period
Monthly Cost	Estimated monthly cost
Recommendation	Keep, Shorten Retention, or Delete
Potential Savings	Estimated savings

Filters: Search (log group name), Region, Unused status, Recommendation

***Tip:** Log groups set to "Never Expire" accumulate storage costs indefinitely. Setting a retention policy is one of the easiest cost wins.*

Fargate Tasks

URL: http://localhost:8000/fargate

Summary Cards: Total Tasks, Spot Eligible, Total Monthly Cost, Total Potential Savings

Details Table:

Column	Description
Task Family	Task definition family name
Cluster / Service	ECS cluster and service name
vCPU / Memory	Allocated compute resources
CPU Utilization	Average and peak CPU (color-coded)
Memory Utilization	Average and peak memory (color-coded)
Uptime	Average hours per day
Restart Count	Restarts in the last month
Spot Eligible	Yes/No badge
Monthly Cost	Estimated monthly cost
Recommendation	Optimal, Downsize, or Migrate to Spot
Potential Savings	Estimated savings

Filters: Search (task family/service), Region, Cluster, Recommendation

ElastiCache

URL: `http://localhost:8000/elasticache`

Summary Cards: Total Clusters, Idle/Oversized Count, Total Monthly Cost, Total Potential Savings

Details Table:

Column	Description
Cluster ID	ElastiCache cluster identifier
Engine	Redis or Memcached
Node Type	e.g., cache.m6g.large
Num Nodes	Number of nodes in the cluster
Multi-AZ	Yes/No badge
CPU Avg / Peak	CPU utilization
Cache Hit Ratio	Percentage of requests served from cache (>90% good, <70% low)
Evictions/sec	Average evictions per second
Connections	Average connection count
Idle	Yes/No badge
Monthly Cost	Estimated monthly cost
Recommendation	Optimal, Downsize, Upsize, or Delete
Potential Savings	Estimated savings

Filters: Search (cluster ID), Region, Engine, Recommendation

Data Transfer

URL: `http://localhost:8000/data-transfer`

Summary Cards: Total Flows, Total Data Transferred (GB), Total Monthly Cost, Total Potential Savings

Details Table:

Column	Description
Traffic Type	Cross-Region, Cross-AZ, NAT→S3, NAT→DynamoDB, or Internet Egress
Source / Dest Region	Origin and destination regions
Data Transferred (GB)	Volume of data in the last month
Monthly Cost	Estimated transfer cost
VPC Endpoint Available	Yes/No — whether a VPC endpoint could replace this flow
Recommended Action	Add VPC Endpoint, Consolidate Regions, or No Action
Potential Savings	Estimated savings

Filters: Search (regions/service), Region, Traffic Type, Recommended Action

***Tip:** VPC endpoints for S3 and DynamoDB are free and eliminate NAT Gateway data processing charges. This is often one of the largest savings opportunities.*

Batch Jobs

URL: <http://localhost:8000/batch-jobs>

Summary Cards: Total Jobs, Spot Eligible, Total Monthly Cost, Total Potential Savings

Details Table:

Column	Description
Job Name	Batch job definition name
Queue / Compute Env	Job queue and compute environment
vCPU / Memory	Allocated resources
Memory Utilization %	How much memory is actually used (color-coded)
CPU Utilization %	How much CPU is actually used (color-coded)
Job Count	Number of runs in the last month
Avg Runtime	Average execution time
Failed Count	Number of failures
Spot Eligible	Yes/No badge
Monthly Cost	Estimated monthly cost
Recommendation	Optimal, Migrate to Spot, or Downsize Memory
Potential Savings	Estimated savings

Filters: Search (job name/compute environment), Region, Recommendation

Lightsail

URL: <http://localhost:8000/lightsail>

Summary Cards: Total Instances, Running Instances, Total Monthly Cost, Total Potential Savings

Details Table:

Column	Description
Instance Name	Lightsail instance name
Bundle ID	Current bundle (e.g., medium_3_0)
Blueprint	OS/application blueprint
State	Running, Stopped
vCPUs / RAM / SSD	Instance resources
CPU Avg / Peak	CPU utilization (color-coded)
Network	Average throughput
Last Active	Date of last activity
Monthly Cost	Bundle cost
Recommendation	Optimal, Downsize, Idle—Stop, or Delete
Potential Savings	Estimated savings

Filters: Search (instance name/bundle), Region, State, Recommendation

11. User Management

URL: `http://localhost:8000/user-management`

This page is available only to users with the **ROOT** or **ADMIN** role. It provides two tabs — **Users** and **Groups** — for managing who can access your Refine server and what accounts they can see.

User Roles

Refine supports three roles:

Role	Description
ROOT	Server administrator. Can manage all users and groups, sees all cloud accounts, and can take any action across all accounts. One per server — the person who runs the setup wizard becomes the ROOT user.
ADMIN	Group administrator. Assigned to groups by Root. Can manage users within their groups, create other admins scoped to their own groups, and take full actions within group accounts — including adding, linking, re-keying and removing those accounts.
USER	Standard user. Assigned to groups by Root or Admin. Works with the resources in group accounts (exempt, modify recommendations, validate an account, etc.). Cannot add, reconfigure or remove cloud accounts, and cannot manage users or groups.

Users Tab

Create and manage user accounts.

Creating a user:

1. Click **Add User**
2. Enter the user's email address and a password
3. Select a role (ADMIN or USER — there is one ROOT per server)
4. Assign the user to one or more groups (determines which cloud accounts they can see). An ADMIN can only assign their own groups.
5. Click **Create User**

Editing a user: Click any user row to change their name, role or active status. Group membership is managed from the **Groups** tab.

Removing a user: Click **Remove** in the Actions column (requires confirmation).

Groups Tab

Groups are named collections of AWS accounts. Users inherit access to all accounts in their assigned groups.

Creating a group:

1. Click **Add Group**
2. Enter a group name (e.g., "Production Team", "Dev Team", "Finance")
3. Select the AWS accounts that belong to this group
4. Click **Create Group**

Editing a group: Click any group row to add/remove accounts or rename the group.

Removing a group: Click **Remove** in the Actions column. Users previously assigned to the group lose access to those accounts unless they belong to another group that includes them.

How Access Works

- The **ROOT** user sees all cloud accounts regardless of group assignment.
- An **ADMIN** or **USER** sees only the accounts included in the groups they belong to — on every page, including inventory, costs, savings, recommendations and exemptions. If a user belongs to multiple groups, they see the union of all accounts across those groups.
- **A user in no group sees no accounts and no data** until someone adds them to a group.
- Organization-wide cost rows that are not attributed to a specific account (from syncs before v3.14) count only for ROOT, because they can't be split by account. A group-restricted user's cost history can therefore start at their accounts' first attributed month.
- Admins can create other admins, but only assign them to groups they themselves own. An admin sees and manages only users who share one of their groups (or are in no group yet) — never the ROOT user.
- An admin can only put accounts they can already see into a group. When an admin adds a new cloud account, it joins the admin's groups automatically so they can see it.
- Opening an account outside your groups by its link returns "not found".
- All users on a Refine server share the same underlying data — groups control visibility, not data collection.

Directory Integration (LDAP / Azure AD)

Refine can integrate with your organization's Active Directory or Azure AD so employees sign in with their corporate credentials instead of separate passwords.

- **On-Prem AD (LDAP):** Employees enter their AD username and password on the Refine login page. Refine authenticates against your AD server directly.

- **Azure AD / Entra ID:** A "Sign in with Microsoft" button appears on the login page. Employees are redirected to Microsoft for authentication and returned to Refine.

Directory users are **auto-provisioned** on first login — no manual account creation needed. AD group memberships can be mapped to Refine account groups, so team members automatically see the right AWS accounts.

The ROOT user always uses local email/password authentication regardless of directory configuration.

To configure: Go to **Directory Settings** in the sidebar (ROOT only). See [LDAP Setup Guide](#) or [Azure AD Setup Guide](#) for step-by-step instructions.

12. System Events

URL: `http://localhost:8000/system-events`

A persistent log of system events — sync failures, S3 read errors, IAM issues, and other operational problems. Each event includes an error code linked to a knowledge base entry explaining the likely cause and recommended fix.

Event List

Column	Description
Severity	Critical (red), Error (amber), Warning (yellow), or Info (blue) badge
Error Code	Machine-readable code (e.g., <code>SYNC_FAILED</code> , <code>S3_READ_ERROR</code>) — click to see the knowledge base entry
Message	Human-readable description of what happened
Source	The service or component that raised the event
Occurrences	How many times this event has been seen (deduplicated)
First Seen	When the event was first recorded
Last Seen	Most recent occurrence
Status	Active, Acknowledged, or Resolved

Actions

- **Acknowledge** — mark an event as seen without resolving it
- **Resolve** — manually close an event
- **Export for Support** — download all events as a JSON file to share with Blacktip Solutions. The export contains only event data (error codes, messages, timestamps) — no credentials or sensitive configuration are included.

Events auto-resolve when the underlying issue is fixed (e.g., a sync that previously failed completes successfully). Repeated occurrences of the same error increment the occurrence count rather than creating separate entries.

13. Common Features

These features are available on every page in Refine.

Sidebar Navigation

The left sidebar provides quick access to all pages:

- **Dashboard** — overview of your whole estate across clouds
- **Cost & Savings** — Cost Overview, Recommendations, Cost Cleanup, Cost Allocation, Commitments, Savings Config, Savings Tracking, Usage Trends, Exemptions
- **Inventory** — Compute, Databases, Storage, Block Storage, Caching, Serverless Functions, Container Workloads, Network, Logs (each page shows all three clouds)
- **Administration** — Cloud Accounts, Users, Directory Settings, System Events (Users and Directory Settings are admin-only)

Click the collapse arrow at the bottom to minimize the sidebar.

AWS Account Selector

If you have multiple AWS accounts connected, use the account selector in the top-right to filter data by account. Select **Deselect All** to see data from all accounts, or check specific accounts to focus on them.

Tag Filtering

Available on inventory pages. Use the tag filter sidebar (bottom-right) to filter resources by tag key and value:

1. Select a tag key from the dropdown
2. Check one or more tag values
3. Click **Add Filter**

Filters use AND logic across keys and OR logic within values. Active filters appear as chips that you can remove individually. Filters persist across page loads.

Sync Now

Click **Sync Now** in the top bar to trigger an immediate data refresh from your AWS accounts. The Lambda function in your AWS account collects data from 14 services in parallel across all regions. First syncs may take a few minutes.

Version and License

The bottom-left corner of the sidebar shows:

- **Refine version** (e.g., Refine vX.Y.Z)
- **License status** — color-coded badge showing your license expiry date

Badge Color	Meaning
Green	License valid
Amber	Expiring soon (within 30 days)
Red	License expired
Purple	Demo mode

14. Cost Allocation

URL: `http://localhost:8000/allocation`

Answers two questions about your tagging: *how much of the estate carries the tags your allocation policy requires*, and *what does each team, project or environment cost*. Both work on the tracked monthly cost of every resource Refine inventories, across all three clouds, and respect the account filter.

Required-tag coverage

Enter the tag keys your policy requires (comma-separated, for example `Project, Environment, Owner`) and click **Apply**. The summary cards show the number of resources in scope, the share that carries every required tag, the number that are untagged or only partly tagged, and the **monthly cost sitting on those untagged resources** — the money you cannot currently allocate. The table underneath lists each required tag with how many resources have it, how many are missing it, and its coverage.

Cost allocation by tag

Choose a tag key (default `Project`) and click **Group**. Each row is one value of that tag with its resource count, tracked monthly cost and share of the tracked total; resources without the tag are grouped as untagged so the column adds up. This is the showback / chargeback view — export the table or read the figures straight into your cost model.

15. Usage Trends

URL: `http://localhost:8000/trends`

Daily utilization history for each resource class, recorded by the nightly sync from CloudWatch (and the Azure equivalent). The page shows one sparkline card per fleet metric — **Compute fleet CPU** (EC2 and Azure VMs), **Databases fleet CPU**, **read and write IOPS**, **Caching fleet CPU and cache hit ratio**, and **Serverless invocations per day and average duration** — each with the p50, p90 and maximum across the fleet for every day of the selected window (**7d**, **30d** or **90d**; the API serves up to a year).

Use it to see whether a fleet's utilization is drifting up or down before you act on a rightsizing recommendation, and to check a single resource's history, which the inventory pages open from the same series. The page reports *No trend data collected yet* until the first sync has run.

16. Directory Settings

URL: `http://localhost:8000/directory-settings` (admin only)

Lets your users sign in with their corporate directory instead of a Refine-local password. Two directories are supported, one active at a time:

- **LDAP / Active Directory** — server URL (`ldaps://...` recommended), user domain, the bind DN and password of a read-only service account, the user search base and filter, the attributes that carry e-mail and display name, the default role for new users (User or Admin) and a connection timeout.
- **Azure AD / Entra ID** — tenant ID, user domain, the client ID and secret of an app registration, and the default role.

Test Connection validates the settings before you save them. **Group Mapping** fetches the directory's groups and lets you map each one to a Refine role and, optionally, to an account group, so access follows directory membership. Directory users are created on their first successful sign-in with the mapped role (or the default role); on every later sign-in their account-group memberships are synced from their directory groups, and their role too unless an admin has locked it on the Users page (see [User Management](#) for how account groups scope what a user sees). Local accounts keep working alongside. **Remove Configuration** switches sign-in back to local passwords only.

Need Help?

Contact Blacktip Solutions at **support@blacktip-ops.com**
